

Optimizing Data Distribution and Kernel Performance for Efficient Training of Chemistry Foundation Models: A Case Study with MACE

Jesun Firoz

Pacific Northwest National Laboratory
USA
jesun.firoz@pnnl.gov

Franco Pellegrini

SISSA
Italy
pellefra@sissa.it

Mario Geiger

NVIDIA
USA
mgeiger@nvidia.com

Darren Hsu

NVIDIA
USA
dahsu@nvidia.com

Jenna A. Bilbrey

Pacific Northwest National Laboratory
USA
Jenna.Pope@pnnl.gov

Han-Yi Chou

NVIDIA
USA
hanyic@nvidia.com

Maximilian Stadler

NVIDIA
USA
mstadler@nvidia.com

Markus Höhnerbach

NVIDIA
USA
mhohnerbach@nvidia.com

Tingyu Wang

NVIDIA
USA
tingyuw@nvidia.com

Dejun Lin

NVIDIA
USA
dejunl@nvidia.com

Emine Kucukbenli

NVIDIA
USA
ekucukbenli@nvidia.com

Henry W. Sprueill

Pacific Northwest National Laboratory
USA
henry.sprueill@pnnl.gov

Ilyes Batatia

University of Cambridge
UK
ib467@cam.ac.uk

Sotiris S. Xantheas

Pacific Northwest National Laboratory
USA
Sotiris.Xantheas@pnnl.gov

Mal-Soon Lee

Pacific Northwest National Laboratory
USA
MalSoon.Lee@pnnl.gov

Christopher Mundy

Pacific Northwest National Laboratory
USA
chris.mundy@pnnl.gov

Gabor Csanyi

University of Cambridge
UK
gc121@cam.ac.uk

Justin S. Smith

NVIDIA
USA
jusmith@nvidia.com

Ponnuswamy Sadayappan

University of Utah
USA
u6024862@gcloud.utah.edu

Sutanay Choudhury

Pacific Northwest National Laboratory
USA
Sutanay.Choudhury@pnnl.gov

Abstract

Chemistry Foundation Models (CFMs) that leverage Graph Neural Networks (GNNs) operating on 3D molecular graph structures are becoming indispensable tools for computational chemists and materials scientists. These models facilitate the understanding of matter and the discovery of new molecules and materials. In contrast to

GNNs operating on large homogeneous graphs, GNNs used by CFMs process a large number of geometric graphs of varying sizes, requiring different optimization strategies than those developed for large homogeneous GNNs. This paper presents optimizations for two critical phases of CFM training: data distribution and model training, targeting MACE - a state-of-the-art CFM. We address the challenge of load balancing in data distribution by formulating it as a multi-objective bin packing problem. We propose an iterative algorithm that provides a highly effective, fast, and practical solution, ensuring efficient data distribution. For the training phase, we identify symmetric tensor contraction as the key computational kernel in MACE and optimize this kernel to improve the overall performance. Our combined approach of balanced data distribution and kernel optimization significantly enhances the training

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of the United States government. As such, the United States Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.

HPDC '25, July 20–23, 2025, Notre Dame, IN, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1869-4/2025/07
<https://doi.org/10.1145/3731545.3731594>

process of MACE. Experimental results demonstrate a substantial speedup, reducing per-epoch execution time for training from 12 to 2 minutes on 740 GPUs with a 2.6M sample dataset.

CCS Concepts

• **Computing methodologies** → **Distributed computing methodologies**; **Neural networks**.

Keywords

Geometric graph neural networks, geometric graphs, Computational chemistry, Foundation models, Batching, Tensor analysis.

1 Introduction

Atomistic simulation serves as the cornerstone of modern computational chemistry and materials science. For these applications, highly accurate quantitative calculations of molecular and material properties are essential. Density Functional Theory (DFT), a quantum mechanical ab-initio (or first-principles) approach, has long been the preferred method for such calculations due to its relatively high generality and accuracy. However, the high time complexity and significant computational resources needed for DFT calculations limit simulations to nanosecond timescales and nanometer length scales [24, 32]. Recently, machine learning interatomic potentials (MLIPs), computational models to map atomic environments to energies and forces, have emerged as attractive alternatives to DFT calculations [21, 24, 40]. The time-to-solution of these surrogate models is significantly shorter while their accuracy is comparable to that of DFT [20].

While traditional specialized surrogate models excel at predicting properties for *specific* chemical systems, the recently-emerging *foundation models* are demonstrating broad capabilities in predicting atomistic properties such as potential energy, forces, stress, and charges for a *diverse* set of chemical species [2, 5, 8, 33, 37, 44]. Foundation models, *trained on large and diverse datasets simultaneously*, are capable of performing a *wide range of general tasks* [9]. The objective of a chemistry foundation model (CFM) is to unify training over a large collection of molecular and materials datasets to improve zero-shot accuracy and facilitate fine-tuning on cases with limited data. When used in zero-shot setting, CFMs can handle a wider range of chemical species than specialized models. During fine-tuning, CFMs reduce the need for extensive data collection and retraining for each new task by leveraging previously learned knowledge from diverse chemical domains [30].

Graph Neural Networks (GNNs) are the dominant architecture for chemistry foundation models (CFMs), with equivariant GNNs favored for their ability to preserve rotational symmetries in atomic systems [7, 35]. However, efficiently capturing many-body interactions (meaning how many neighboring atoms/vertices are considered simultaneously to compute contribution to per atom energy) —a critical aspect of modeling chemical systems—remains a challenge due to the high computational costs of operations on higher-order tensors. To address this, the atomic cluster expansion (ACE) formalism [12] was developed to describe many-body interactions efficiently. Building on ACE, Batatia et al. [6] introduced MACE, which integrates these interactions into equivariant message-passing schemes, improving computational efficiency

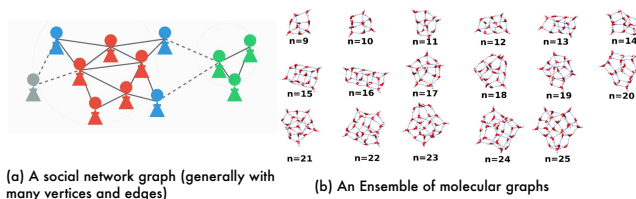


Figure 1: Key contrast between different classes of GNNs: Molecular GNNs are trained on many small graphs instead of a single big graph.

without sacrificing accuracy. MACE extends beyond pairwise interactions to capture complex multi-atom effects and has been widely adopted as the state-of-the-art CFM for molecular and materials modeling [4–6, 23].

Key Traits of CFM Training Workloads. CFMs present distinctly different scalability challenges compared to conventional GNN workloads (cf. Figure 1 and Table 1), which typically focus on single massive graph [18, 25, 41]. Instead, CFMs train on collections of small, irregular 3D molecular graphs, requiring careful data distribution to avoid load imbalance and straggler effects across GPUs. These molecular graphs vary in size, sparsity, and chemical composition, making efficient batching and workload partitioning a nontrivial problem.

Another fundamental trait of CFM workloads is the dominance of symmetric tensor contractions—high-cost operations central to equivariant GNNs. These tensor operations scale poorly without specialized optimizations, making them a primary bottleneck in large-scale training. Given that CFM training can require anywhere from ≈ 3000 [5] to ≈ 34000 GPU-hours [37], optimizing tensor computations and load balancing is critical for scalability.

Why is MACE a representative CFM workload? Many state-of-the-art equivariant GNNs, including MACE and NequIP [7], share a common mathematical foundation, where different models correspond to specific parameter choices within a unified design space [4]. This commonality highlights shared computational challenges, particularly the high cost of symmetric tensor contractions, making MACE an ideal target for optimization.

Beyond its architectural significance, MACE has demonstrated remarkable generalization across diverse chemical systems. Unlike traditional machine-learned force fields that require system-specific training, MACE has been successfully applied to a broad range of atomistic simulations, including solids, liquids, gases, chemical reactions, and interfaces [4–6]. This broad applicability positions MACE as a strong candidate for a foundation model in chemistry, reinforcing the importance of optimizing its computational efficiency. Therefore, by focusing on MACE, we ensure that our optimizations extend to a wider class of CFMs built on equivariant GNNs, making them broadly impactful for large-scale atomistic modeling.

Contributions. In summary, we make the following contributions in this paper.

- We formulate the problem of efficient data distribution of 3D molecular graphs as a multi-objective bin packing problem with two objectives: achieving balanced data distribution while reducing the memory footprint of padding. We propose a practical, fast, and effective iterative algorithm to tackle this problem (Section 3).
- We identify the key computational kernel, symmetric tensor contraction, that are common operations across multiple equivariant

Aspect	GNNs for large Graphs	GNNs for Geometric/Molecular Graphs
Input graph structure	Single massive graph	Collection of many smaller graphs
Node count	Millions to billions	Typically <1000 per graph
Data distribution	Usually a single connected graph that needs to be partitioned across workers (partition vertices and edges)	Natural partitioning as each molecular graph is independent. Each worker processes multiple graphs in parallel.
Node features	Typically scalar features only	Mix of scalar (atom type) and geometric features
Symmetries	Only permutation symmetry needs to be preserved	Multiple symmetries: permutation, rotation, translation
Edge definition	Fixed edges based on relationships	Dynamic edges often based on distance cutoffs between atoms
Key computations	Graph partitioning, neighborhood sampling, feature caching	Tensor product and contraction, message passing
Performance bottleneck	Communication overhead of node features between workers	Tensor computations and geometric feature calculations

Table 1: Comparison between GNNs for large graphs vs. geometric/molecular graphs.

GNNs, and propose optimization techniques for the symmetric tensor contraction kernel (Section 4).

- We train a GNN-based CFM on a diverse dataset of 2.6M molecular configurations, spanning 1–768 atoms and varying sparsity profiles, to study the impact of non-uniform dataset composition on load balancing and training efficiency (Section 5.1.1).
- Our evaluation demonstrates a speedup of 6× for the state-of-the-art CFM MACE on 740 GPUs with the dataset containing 2.6M molecular structures, with wide range of diverse molecules, both in terms of the size and sparsity pattern (Section 5).

2 Background

Molecules can be naturally represented as 3D geometric graphs, where atoms are vertices and bonds are edges, with additional geometric attributes such as atomic coordinates, angles etc. Therefore, molecular GNNs need to account for spatial information such as atomic coordinates and distances, and respect physical symmetries. Molecular structures can be altered through rotations, reflections, or translations without changing their fundamental structure or properties. *Invariant GNNs* [10, 15, 16, 36] produce outputs that are independent of the molecule’s orientation, while *equivariant GNNs* (EGNNs) [6, 7, 26, 29, 39] preserve the relationship between input features and their corresponding hidden representations under symmetry transformations like rotations. Also, for molecular GNNs the message passing mechanism is adapted to incorporate both topological and geometric attributes. The next two sub-sections provide necessary background on these two key aspects.

2.1 Equivariant Molecular Representations: Motivation

In molecular modeling, atoms interact in three-dimensional space, and their properties should change predictably when the molecule is rotated. This is crucial for tasks like energy prediction and force calculations, where rotational symmetry plays a fundamental role. Equivariant representations ensure that a model’s understanding of atomic environments transforms correctly under rotation—preserving the underlying physics. Models such as MACE learns equivariant representations using the mathematical framework of group theory and spherical harmonics to describe atomic environments while naturally handling rotations.

Group theory [42] provides a mathematical framework for describing *symmetries* in physical systems. In the context of molecular modeling, the $SO(3)$ group represents all possible 3D rotations. A *representation* of $SO(3)$ describes how objects transform under these rotations, and *irreducible representations* (irreps) are the fundamental building blocks that cannot be decomposed further.

Spherical harmonics are special mathematical functions that describe how properties vary on the surface of a sphere and describe how atoms interact in 3D space while naturally preserving rotational properties. Spherical harmonics are typically denoted as $Y_l^m(\hat{\mathbf{r}})$, where l is the *degree/order* ($0, 1, 2, \dots$), m is the *azimuthal order/phase* ($-l \leq m \leq l$) and $\hat{\mathbf{r}}$ is a vector lying on the unit sphere. Spherical harmonics of degree l form a basis for the $(2l+1)$ -dimensional irreducible representation of $SO(3)$, the 3D rotation group. This property makes them ideal for constructing rotationally equivariant neural network layers.

2.2 Learning Equivariant Representations via Message-Passing Architecture

MACE [6] is an equivariant message-passing GNN where each layer encodes many-body information of atomic geometry. A schematic diagram of the MACE model is shown in Figure 2 (a-d). In MACE, each atom i is associated with its position $\mathbf{r}_i$, the atomic numbers z_i , and a set of equivariant learnable features $h_{i,klm}$, where k denotes the number of the learnable channels, l and m relates to the order of the spherical harmonics.

First, the *embedding layer* (Figure 2(b)) creates a set of features for the nodes and edges describing the chemical species and the length and orientation of each edge. This is done by encoding the interatomic displacements ($\mathbf{r}_{ji}$) in a set of radial basis functions and spherical harmonics $Y_l^m(\hat{\mathbf{r}}_{ji})$, where $\hat{\mathbf{r}}_{ji}$ is the unit vector that points from a neighboring atom j to atom i .

Next, the *interaction layer* pools information from the neighbors (Figure 2(c)) and computes per-atom features $A_{i,klm}$ of the surrounding environment from the radial $R_{l_1,l_2,l_3}(r_{ji})$, spherical $Y_l^m(\hat{\mathbf{r}}_{ji})$, and per atom $h_{i,klm}$ embeddings, while maintaining equivariance of these features using the Clebsch-Gordan Coefficients.

The *product layer* (Figure 2 (d)) then raises the information aggregated from the neighbors to a power to create high body-order features $B_{i,\eta,\nu,kLM}$ from the $A_{i,klm}$ features. This is done by taking a product of the $A_{i,klm}$ up to a correlation order ν . This product mixes the various equivariance levels of $A_{i,klm}$, each encoding different aspects of the local chemical environment, to produce a feature that encodes up to ν atom interactions.

Following the framework of message passing GNN, message at node i for iteration step t is then constructed as

$$m_i^{(t)} = \bigoplus_{j \in N(i)} M_i(\sigma_i^{(t)}, \sigma_j^{(t)}) \quad (1)$$

where $N(i)$ is the neighborhood of node i , M_i is a learnable message function, and $\bigoplus_{j \in N(i)}$ is the *aggregation* operation, σ_i is the state of node i . In MACE, for channel k , spherical harmonic order L , M

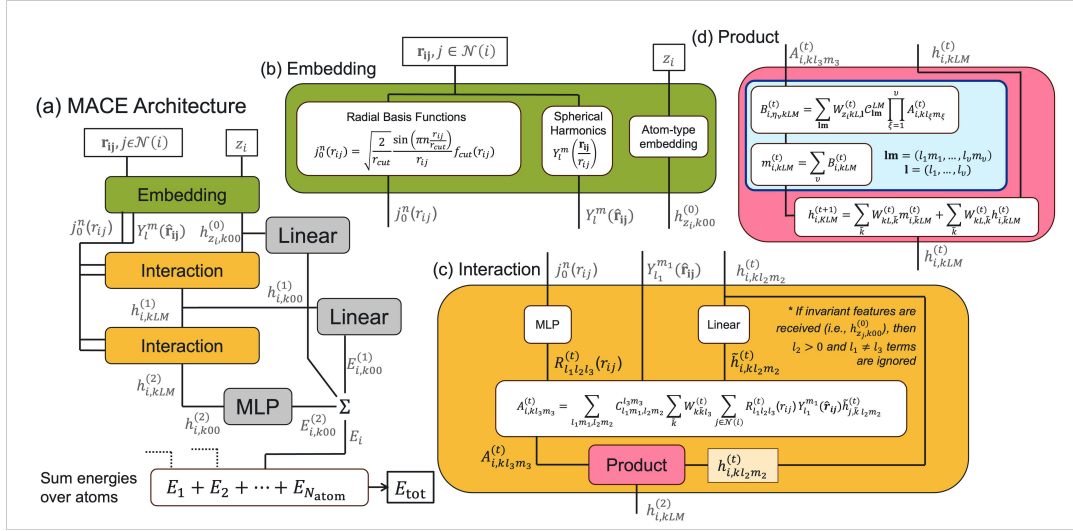


Figure 2: A schematic diagram of the MACE model (a-d) [6]. In (a), input and output tensor are shown for each of the embedding, interaction, linear readout, and MLP readout layers. Two interaction layers are shown as this is the number typically used in MACE. The construction of the A_{i,k,l_3,m_3} is shown in the interaction layer (c) with details of the higher body-order products shown in (d). Algorithm 3 shows the details for computing the product in the blue box of (d).

Table 2: List of symbols and notation

Symbol	Description
$Y_{ji,l}^m$	Spherical harmonics of order l , component m for edge vector ji
r_{ji}	Interatomic displacement vector between atoms j and i
$\hat{r}_{ji}$	Unit vector pointing from atom j to atom i
$h_{j,klm}^{(t)}$	Learnable equivariant features at iteration t for atom j , channel k , order l , component m
$A_{i,klm}$	Atomic basis features
$C_{l_1 m_1, l_2 m_2}^{l_3 m_3}$	Clebsch-Gordan coefficients for combining spherical harmonics of orders (l_1, m_1) and (l_2, m_2) into order (l_3, m_3)
C_{lm}^{LM}	Generalized Clebsch-Gordon coefficients
$R_{ji,kl_1 l_2 l_3}^{(t)}$	Weights at iteration t for edge ji , channel k , and spherical harmonic orders l_1, l_2, l_3
$W_{z_i,kl,l}^{(t)}$	Learnable weights at iteration t dependent on atomic species z_i , channel k , orders L, l
z_i	Atomic numbers
$v_{\max}$	Maximum correlation order
$L_{\max}$	Maximum irreducible representation order

and learnable weights W , the message is computed as

$$m_{i,k,L,M}^{(t)} = \sum_v \sum_{\eta_i} W_{k,\eta_i}^{(v)} B_{\eta_i,k,L,M}^{(v,s)} \quad (2)$$

The inner sum considers all possible combinations of (l, m) and this is done up to a certain body order v . The per-atom features are then updated using a residual update from these higher body-order features and the previous $h_{i,klm}$.

At each intermediate interaction layer, the invariant part ($h_{i,k00}$) of the atom-wise feature is passed through a linear layer and pooled to produce an energy contribution. Only the final feature is passed through an MLP to compute its energy contribution. Forces are calculated by taking derivatives of the total energy. Table 2 summarizes the notation and symbols.

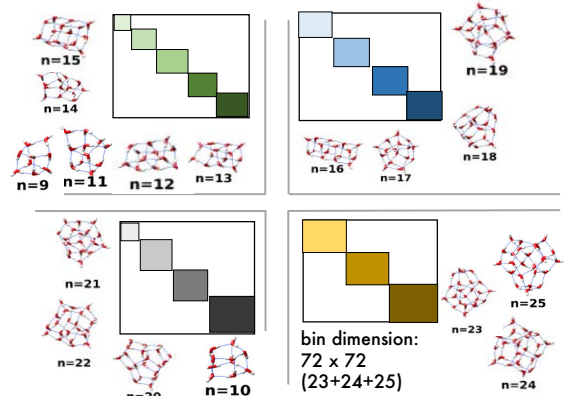


Figure 3: Minibatch creation and distribution process of molecular graphs for a GNN model training across 4 compute nodes.

3 Optimizations for Data Distribution in CFMs

Challenges in distributing molecular graphs for equivariant GNN training. A Chemistry foundation model is typically trained on a large set of molecular graphs drawn from various chemical systems. These graphs are generally small but diverse in terms of the vertex counts and the edge counts (Table 3 and Figure 5). CFMs implemented with popular graph neural network libraries such as PyTorch Geometric (PyG) [13] and Distributed Deep Graph Library (DGL) [43] often combine a fixed number of these graphs into a single graph to create *mini-batches* to process the graphs simultaneously. This is done by placing the adjacency matrices of the individual molecular graphs along the diagonal, creating a large graph with isolated subgraphs (for example, the colored blocks placed within each square in Figure 3). Once these mini-batches are created, they are distributed across multiple GPUs to train a model.

Popular frameworks like PyG employ *fixed-size* mini-batching i.e. *fixed number of graphs combined in a mini-batch* (for example 5 graphs per minibatch irrespective of their sizes). However, a straightforward approach of randomly combining graphs have limitations.

Observation 1: Using a fixed number of molecular graphs in each mini-batch ignores the diversity in graph sizes that can lead to uneven sizes in mini-batches, impacting performance. Based on the sparsity pattern in each graph, the total amount of work in each mini-batch can also vary significantly. Consequently, when the workload is distributed across multiple GPUs, the GPU execution time can differ, with the slowest GPU (straggler) limiting overall performance.

3.1 Mini-batches Creation as the Multi-objective Bin Packing Problem

We address the challenge of efficient distribution of molecular graphs by formulating it as a multi-objective bin packing problem [22, 34]. Given a list of graph vertex counts and a maximum bin capacity, our goal is to create balanced batches that minimize both load imbalance and memory padding overhead, while ensuring the number of batches is a multiple of available GPUs for efficient distributed training. We develop an efficient iterative algorithm to address this challenge. This problem can be approached as either a scheduling problem with a fixed number of bins or a bin packing problem where the number of bins is determined by the algorithm [1]. As an example, the distribution scheme in Figure 3 employs four bins (represented with large squares) to store the molecular graphs, with the bins containing 5, 4, 3, and 4 molecular graphs in clockwise order. The bottom-right bin contains three molecular graphs with 23, 24, and 25 vertices, yielding a total *token count* (combined vertex count) of 72 (dimension of the bin).

3.1.1 Problem formulation. Let us denote the number of vertices in a graph with vertex set V by $|V|$. We assume there are N graphs to be distributed across the mini-batches (bins). The maximum number of possible bins is N , and we define binary variables a_j to indicate whether bin j is used ($a_j = 1$) or not ($a_j = 0$). This variable a_j is subject to minimization. The assignment variable b_{ij} indicates whether the i -th graph is placed in bin j ($b_{ij} = 1$), otherwise $b_{ij} = 0$. The capacity of each bin i.e. the maximum number of vertices is denoted by C . The width and the height of each (square) bin is $W(=C)$. We want to find a $t \in \mathbb{N}$ and assignment $f: \{1, \dots, N\} \rightarrow \{1, \dots, t\}$ with objective functions

$$\min \sum_{j=1}^N a_j \quad (3)$$

$$\min \sum_{j=1}^t \left[\sum_{i=1}^N \frac{b_{ij}|V_i|^2}{W^2} \right] \quad (4)$$

$$\min \max_{j,k} \left| \sum_{i=1}^N (b_{ij}|V_i|^2) - \sum_{i=1}^N (b_{ik}|V_i|^2) \right| \quad (5)$$

Algorithm 1: CREATE-BALANCED-BATCHES

Input: $L = (l_1, \dots, l_N)$: list of graph vertex counts, C : bin capacity, G : number of GPUs
Output: $\mathcal{B}$: set of bins with graph indices

```

1  $(L, I) \leftarrow \text{StableSort}(L, \text{descending} = \text{True})$  //  $I$  is the index mapping
2  $S \leftarrow \sum_{i=1}^N l_i$ 
3  $M \leftarrow \left\lceil \frac{S}{C} \right\rceil$ 
4  $M \leftarrow \left\lceil \frac{M}{G} \right\rceil \cdot G$ 
5  $\mathcal{B} \leftarrow \{B_1, \dots, B_M\}$ , where  $\forall j, B_j = \emptyset$  and  $c(B_j) = C$ 
6  $p \leftarrow 1$  // Pointer to current item in  $L$ 
7 while  $p \leq N$  and  $\mathcal{B} \neq \emptyset$  do
8    $\mathcal{B} \leftarrow \text{StableSort}(\mathcal{B}, \text{key} = c, \text{descending} = \text{True})$ 
9   for  $j \leftarrow 1$  to  $|\mathcal{B}|$  do
10    if  $c(B_j) \geq l_p$  then
11       $B_j \leftarrow B_j \cup \{I_p\}$ 
12       $c(B_j) \leftarrow c(B_j) - l_p$ 
13       $p \leftarrow p + 1$ 
14      if  $p > N$  then
15        break
16    else
17      Mark  $B_j$  as full
18   $\mathcal{B}_{\text{full}} \leftarrow \{B_j \in \mathcal{B} : B_j \text{ is full}\}$ 
19   $\mathcal{B} \leftarrow \mathcal{B} \setminus \mathcal{B}_{\text{full}}$ 
20  if any non-full bin has less remaining capacity than a full bin then
21    Unmark all full bins in  $\mathcal{B}_{\text{full}}$ 
22     $\mathcal{B} \leftarrow \mathcal{B} \cup \mathcal{B}_{\text{full}}$ 
23  if  $p \leq N$  then
24     $\mathcal{B}_{\text{remaining}} \leftarrow \text{CREATE-BALANCED-BATCHES}((l_p, \dots, l_N), C, G)$ 
25     $\mathcal{B} \leftarrow \mathcal{B} \cup \mathcal{B}_{\text{remaining}}$ 
26 return  $\mathcal{B}$ 

```

$$\sum_{i=1}^N |V_i| b_{ij} \leq C a_j \quad \forall j. \quad (6)$$

$$\text{s.t.} \quad \sum_{j=1}^t b_{ij} = 1 \quad \forall i \quad (7)$$

Equations (3) to (5) states the following objectives: we aim to minimize the number of bins, while minimizing zero-padding memory in each bin (mini-batch), and the difference between the amount of space filled by molecular graphs in any two bins should be minimal. Equation (6) is the capacity constraint. The assignment constraint (Equation (7)) ensures that each graph is assigned to a bin. Given a fixed bin capacity, padding refers to the unused memory that must be zero-filled when no remaining graph fits in the available space.

3.2 Iterative Algorithm for Multi-Objective Bin Packing

The multi-objective bin packing for molecular graphs is a NP-hard problem [14]. To address the multi-objective bin packing problem, we propose an iterative algorithm (Algorithm 1) that balances computational load while minimizing padding overhead. The algorithm takes as input a set of graphs with associated vertex counts, a maximum capacity per bin, and the number of GPUs used. The objective is to distribute these graphs across M bins, a multiple of the number of GPUs, in a way that balances the load across GPUs while respecting capacity constraints.

Initially, graphs are sorted by their vertex count in descending order to handle larger graphs first (Line 1). In each iteration, bins are stably sorted by their remaining capacity to prioritize fuller bins (Line 8). Within the iteration, the algorithm steps through all (remaining) bins once (Line 9). The algorithm iteratively assigns a graph to each bin. If a bin’s remaining capacity is higher than the size of the graph currently considered (Line 10), the corresponding index is appended to the bin (Line 11), the remaining capacity reduced (Line 12), and the pointer advances (Line 13). When a bin cannot accommodate the current item, it is marked as full (Line 17). At the end of the iteration, the full bins are collected (Line 18). Intuitively, the process can be thought of as the cyclic distribution of the sorted molecular graphs across the available bins.

However, the algorithm also incorporates an adaptive bin management mechanism: when a bin marked as “not full” achieves a higher occupancy than bins previously marked as “full”, it indicates we have succeeded in packing more items than was deemed possible earlier. At this point, all bins previously marked as ‘full’ are reconsidered and returned to the active pool (Lines 20–22), as they may now be able to accommodate the currently smaller remaining graphs. This opportunistic reassessment of bin availability helps achieve higher packing density across all bins.

This process continues until all graphs are assigned or no suitable bins remain. If any graphs are left unassigned, the process is repeated for remaining graphs (Lines 23–25). Stable sorting is used to maintain consistency across processes. The algorithm produces locally uniform batches, resulting in balanced load across GPUs.

Note that, our algorithm differs from well-known best-fit and first-fit algorithms [14] for bin packing in several ways. First, methods like best-fit algorithm aims to minimize waste in each individual bin placement. In contrast, our proposed algorithm optimizes for both bin utilization and load balance across adjacent bins/GPUs. Our algorithm distributes the graphs among the bins, compared to the best-fit algorithm. The later fills in a bin until full and then opens a new bin. Additionally, our algorithm pre-processes all molecular graphs by sorting them by vertex counts and has a global view of the workload optimization.

3.2.1 Implementation details. We have modified PyTorch’s Distributed Sampler to implement our algorithm. The sampler is edited to become a batch sampler, where at the beginning of every epoch, the batches are determined with Algorithm 1. Note that the “size” of a graph can be defined arbitrarily, in addition to above-mentioned vertex count, such as edge count, or a function of both.

3.2.2 Time complexity. The initial sorting of graphs takes $O(N \log N)$, where N is the total number of graphs. In each iteration, sorting the remaining capacity of bins takes $O(M \log M)$, where M is the number of bins. The number of iterations is $O(\frac{N}{M})$. Therefore the overall time complexity of the algorithm is $O(N \log N) + O(N \log M)$. On a single CPU, with $\sim 1M$ molecular graph samples and $\sim 100k$ batches, preparing the batches with this algorithm takes about one second.

4 Computational Kernel Optimization

We now focus on the most computationally expensive operations within the MACE architecture’s interaction layer and the product block (Figure 2 c-d): the symmetric tensor contraction and the tensor

product to construct the messages, referred to as the channelwise tensor product (TP). Tensor contraction serves as the fundamental operation in MACE [6] and other prominent models such as NequIP [7], Allegro [29] and Cormorant [3].

4.1 Symmetric Tensor Contraction: Optimization Needs

Algorithm 2: Construction of original messages in MACE

Input: $C_{l_1 m_1, l_2 m_2}^{l_3 m_3}$: Clebsch-Gordan coefficients
Input: $R_{ji, kl_1 l_2 l_3}^{(t)}$: Precomputed weights for edge ji
Input: $h_{j, klm}^{(t)}$: Features of atom j
Input: $Y_{ji, l}^m$: Spherical harmonics for edge ji
Input: N_{edges} : Number of edges
Input: N_{channels} : Number of channels
Input: L_{max} : Maximum irreps order

```

1 for  $ji = 1$  to  $N_{\text{edges}}$  do
2   for  $k = 1$  to  $N_{\text{channels}}$  do
3     for  $l_3 = 0$  to  $L_{\text{max}}$  do
4        $A_{ji, kl_3 m_3}^{(t)} \leftarrow 0$ 
5       for all combinations of  $l_1 m_1, l_2 m_2$  do
6          $A_{ji, kl_3 m_3}^{(t)} \leftarrow A_{ji, kl_3 m_3}^{(t)} + C_{l_1 m_1, l_2 m_2}^{l_3 m_3} R_{ji, kl_1 l_2 l_3}^{(t)} Y_{ji, l_1}^{m_1} h_{j, kl_2 m_2}^{(t)}$ 

```

Output: $A_{ji, kl_3 m_3}^{(t)}$

4.1.1 Bottleneck in atomic basis (A) construction (Algorithm 2). In MACE, messages are constructed from vertex features and the relative vertex positions (the interaction block in Figure 2 (c)). The first step to build messages is the channelwise TP (Figure 4) presented in Algorithm 2, Line 6. Let us consider the message from atom j to atom i at epoch t : the first factor entering in the message are vertex features $h_{j, klm}^{(t)}$, which represent properties of the sender atom (vertex) in channel k and of order l . The second factor is the relative position of atoms i and j expressed in terms of spherical harmonics $Y_l^m(\hat{r}_{ji}) = Y_{ji, l}^m$. The third factor, the learnable weights, $R_{ji, kl_1 l_2 l_3}^{(t)}$, are pre-computed from a multilayer perceptron taking a radial embedding of the interatomic distances r_{ji} as input, and determine how much importance is given to different types of interaction. This algorithm performs a convolution-like operation: it combines information from the vertex features $h_{j, klm}^{(t)}$ and the geometric relationship between vertices $Y_{ji, l}^m$ using learnable weights and the Clebsch-Gordan coefficients, $C_{l_1 m_1, l_2 m_2}^{l_3 m_3}$ (Lines 5-6).

This step is particularly expensive because the product is computed for each edge of the graph. The parameter L_{max} limits the maximum order of the resulting message and can be tuned to obtain a higher order (but more expensive) description.

Unexploited sparsity in Clebsch-Gordan coefficient access. In Algorithm 2, when constructing messages via Clebsch-Gordan coefficients $C_{l_1 m_1, l_2 m_2}^{l_3 m_3}$, only specific combinations of (l_1, m_1, l_2, m_2) yield non-zero coefficients. The number of non-zero elements typically represents less than 20% of possible coefficient combinations. This sparsity pattern is deterministic and known at compile time.

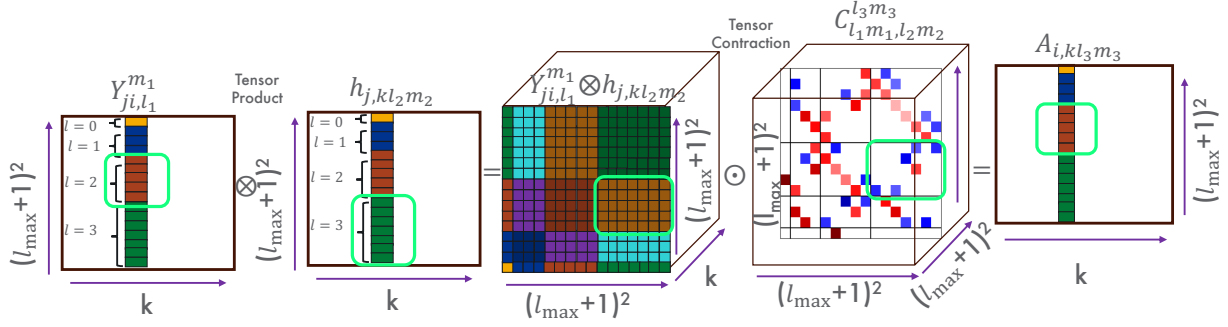


Figure 4: A schematic diagram of the tensor contraction performed in Algorithm 2. Conceptually, two equivariant tensors, $Y_{ji,l_1}^{m_1}$ and h_{j,kl_2m_2} , are multiplied via the normal tensor product to form the product tensor $Y_{ji,l_1}^{m_1} \otimes h_{j,kl_2m_2}$ which is contracted using the Clebsch-Gordan coefficients to form the output equivariant tensor A_{i,kl_3m_3} . The additional channel dimension k is mixed by the radial embedding $R_{ji,kl_1l_2l_3}$ (not shown). In practice the product is computed for every combination of l_1, l_2, l_3 up to $l_{\max}$. The green boxes depict the $l_1 = 2, l_2 = 3, l_3 = 2$ selection and the Clebsch-Gordan coefficient matrix is shown for $l_1 = 2, l_2 = 3, l_3 = 2$ and $m_3 = -1$, which is highly sparse.

The current approach does not take CG tensor’s sparsity into consideration and applies dense matrix multiplication.

Observation 2: The existing approach doesn’t consider the sparsity of Clebsch-Gordan coefficients, leading to unnecessary computations in dense matrix multiplication, while exploiting these properties could significantly reduce storage and computational requirements.

Algorithm 3: Higher-order feature computation in MACE

Input: $A_{i,klm}^{(t)}$: Atomic basis features
Input: C_{lm}^{LM} : Generalized Clebsch-Gordan coefficients
Input: $W_{zkl,l}^{(t)}$: Learnable weights
Input: N_{atoms} : Number of atoms
Input: N_{channels} : Number of channels
Input: $L_{\max}$: Maximum irreps order
Input: $v_{\max}$: Maximum correlation order

```

1 for i = 1 to N_atoms do
2   for k = 1 to N_channels do
3     for L = 0 to L_max do
4        $m_{i,kLM}^{(t)} \leftarrow 0$ 
5       for v = 1 to v_max do
6         for all combinations lm of  $l_1m_1, \dots, l_v m_v$  do
7           prod  $\leftarrow 1$ 
8           for  $\xi = 1$  to v do
9             prod  $\leftarrow$  prod  $\cdot A_{i,kl_\xi m_\xi}^{(t)}$ 
10             $m_{i,kLM}^{(t)} \leftarrow m_{i,kLM}^{(t)} + W_{zkl,l}^{(t)} C_{lm}^{LM} \cdot \text{prod}$ 

```

Output: $m_{i,kLM}^{(t)}$

4.1.2 Bottleneck in message construction i.e. “pooling” (Algorithm 3). After messages from all neighbors j are received, they are summed over j (by channel k and order l) on the receiving vertex i . After a further linear combination (between terms k of the same order), the central operation of MACE can be performed: the symmetric tensor contraction (Line 10 in Algorithm 3). In this operation, multiple

copies of the message are contracted (through a tensor product) in order to produce features of higher order (Figure 2 (d)).

As detailed in Algorithm 3, a product of up to v copies of the messages $A_{i,klm}^{(t)}$ are computed, for a given channel k and considering all possible combinations of $l_1m_1, \dots, l_v m_v$ that would result in a nonzero contribution to a given LM , up to $L_{\max}$ (Lines 5-9). Similar to Algorithm 2, the high sparsity of the C_{lm}^{LM} tensor makes this symmetric tensor contraction particularly attractive for optimization.

Inefficiency in symmetric tensor contraction implementation in existing libraries. Tensor contractions, specifically symmetric tensor contractions in the MACE model are originally implemented via contracting multiple tensor segments as a chain of small kernels in auto-differentiation deep learning frameworks such as PyTorch-based e3nn [17]. For tensor contractions, features for a tuple (l, m) correspond to tensor segments which are then naively contracted for each (l_1, l_2, l_3) or even for each $(l_1, m_1, l_2, m_2, l_3, m_3)$ in separate kernel calls. This results in inefficiency in modern GPUs. Naively chaining these primitives thus results in repeated loads and stores of intermediate results into the global memory.

Observation 3: Existing implementations in frameworks like PyTorch e3nn perform symmetric tensor contractions by breaking them into many small separate kernel calls for each combination of quantum numbers (l, m) , leading to excessive global memory access, poor GPU utilization, and frequent small kernel launch overhead.

4.2 Optimization Strategies

To address the inefficient symmetric tensor contractions and unexploited Clebsch-Gordan coefficient sparsity patterns identified above in Pytorch e3nn library, that provides the building blocks for many well-known equivariant GNNs including MACE, and to improve the overall performance of Algorithm 2 and Algorithm 3, we apply the following optimization techniques. Due to space constraint, we only provide highlighted optimizations in code for Algorithm 3 in Listing 1.

4.2.1 Applying kernel fusion. To avoid the problem of launching multiple small kernels, *kernel fusion* combines as many of these operations in a single GPU kernel thus allowing for keeping intermediate results in shared memory as long as possible. Shared memory is faster by several orders of magnitude, thus improving the overall performance. We adopt the kernel fusion technique for the symmetric tensor contraction, replacing many natively-supported operations with a single special-purpose GPU kernel. Kernel fusion executes the entirety of Algorithm 2 and Algorithm 3 in a single kernel with parallelism derived from their outermost loops. By applying kernel fusion technique to combine multiple small kernels in a larger GPU kernel improves kernel launch time, enables compilers and machine learning frameworks to automatically detect and fuse kernels and operators.

However, there are several challenges associated with implementing kernel fusion. Specifically, kernel fusion requires careful balancing of resource usage while extracting performance. The main challenge is resource exhaustion and reduced parallelism. While fused kernels require more resources (registers, shared memory, instruction cache) than the unfused ones, we mitigate this through techniques like reusing the shared memory buffers (❶ in Listing 1). Our molecular graph domain provides sufficient parallelism through the number of edges and vertices to maintain efficient execution despite these resource constraints.

4.2.2 Exploiting Clebsch-Gordan (CG) tensor sparsity. According to the law in quantum mechanics, there is a set of key rules that determine which combinations give non-zero CG coefficients. Specifically, for $C_{l_1 m_1, l_2 m_2}^{l_3 m_3}$, the l values must satisfy the “triangle rule”: $|l_1 - l_2| \leq l_3 \leq l_1 + l_2$. The m values must satisfy: $m_1 + m_2 = m_3$, $|m_1| \leq l_1$, $|m_2| \leq l_2$, and $|m_3| \leq l_3$. Since these rules are known at compile time, we can pre-compute all valid combinations, store only non-zero coefficients, and create lookup tables for fast access. Additionally, CG-tensor sparsity optimization converts operations on the CG tensor into sparse multiplications focusing only on non-zero elements (❷ in Listing 1).

4.2.3 Maximizing bandwidth utilization. Memory bandwidth is optimized through efficient thread-layouts for coalesced access patterns and vectorized load/store operations. In particular, for Algorithm 2, the implementation can utilize vectorized loads through float4 operations to efficiently load *blocks* of the input tensors R and h and coalesced memory access so that adjacent threads access adjacent memory locations. Similarly, in Algorithm 3, the input features A can be loaded using vectorized operations and stored in shared memory (❷ in Listing 1). In addition, shared memory can be utilized by storing frequently accessed data such as the Clebsch-Gordan coefficients and spherical harmonics Y . Shared memory utilization reduces global memory access by preloading and operating on data in shared memory.

4.2.4 Utilizing warp-level primitives and loop unrolling. For Algorithm 2, the computation is organized such that each thread block handles a specific (l_3, m_3) combination, with warps collaboratively processing different (l_1, m_1, l_2, m_2) combinations. Within each warp, threads exploit the regular structure of the indices by distributing the m_2 indices across threads while sharing the same l_2 value, enabling efficient parallel reduction using warp shuffle.

```

1 __global__ void optimized_higher_order_features(
2     const float* A_t, const float* CG_LM, const float* W_t,
3     float* m_t, const int* z_i, const int I, const int K,
4     const int L_max, const int M_max, const int nu_max) {
5     // Shared memory for input features and intermediate results
6     __shared__ float s_A[BLOCK_SIZE][MAX_L][MAX_M]; //❶ Input features
7     __shared__ float s_B[MAX_NU][MAX_ETA][MAX_K]; //Intermediate res.
8
9     const int tid = threadIdx.x;
10    const int wid = tid / WARP_SIZE; const int lid = tid % WARP_SIZE;
11    const int atom_idx = blockIdx.x; // Each block handles one atom
12    const int L = blockIdx.y; const int M = blockIdx.z;
13
14    //Load atomic features into shared memory(vectorized and coalesced)
15    for(int k = wid; k < K; k += gridDim.x) {
16        float4 a_vals = *reinterpret_cast<const float4*>(
17            &A_t[atom_idx * K * L_max * M_max + k * L_max * M_max]); //❷
18
19    #pragma unroll 4
20    for(int idx = 0; idx < 4; idx++) {
21        int l = (lid * 4 + idx) / M_max;
22        int m = (lid * 4 + idx) % M_max;
23        if(l < L_max && m < 2*l + 1) {
24            s_A[tid][l][m] = ((float*)a_vals)[idx];
25        }
26    }
27    __syncthreads();
28
29    // Process each correlation order
30    float result = 0.0f;
31    for(int nu = 1; nu <= nu_max; nu++) {
32        // Each warp processes different combination of indices
33        float eta = wid; eta < num_coupling_patterns[nu]; eta += gridDim.x {
34            float prod = 1.0f;
35            // Process tensor products within warp
36            #pragma unroll
37            for(int xi = 0; xi < nu; xi++) {
38                // Get indices for this correlation order
39                int l_idx = get_l_index(nu, eta, xi);
40                int m_idx = lid; // Distribute m indices across warp
41                if(m_idx < 2*l_idx + 1) {
42                    // Load and multiply feature values
43                    float a_val = s_A[tid][l_idx][m_idx];
44                    // Warp-wide product using shuffle
45                    //❸ #pragma unroll
46                    for(int offset = WARP_SIZE/2; offset > 0; offset /= 2) {
47                        float other = __shfl_xor_sync(0xffffffff, a_val, offset); //❹
48                        a_val *= other; }
49                    // First thread in warp accumulates
50                    if(lid == 0) prod *= a_val;
51                }
52                // Get corresponding CG coefficient sparse index
53                float cg = CG_LM[get_cg_index(nu, eta, L, M)]; //❺
54                int z = z_i[atom_idx];
55                float weight = W_t[get_weight_index(z, K, L, eta)];
56                // Accumulate contribution
57                if(lid == 0) { // First thread in warp
58                    float contrib = cg * weight * prod;
59                    // Atomic add to handle concurrent updates
60                    atomicAdd(&m_t[get_output_index(atom_idx, K, L, M)], contrib);
61                }
62            }
63        }
64    }
65 }

```

Listing 1: Optimized Algorithm 3

For Algorithm 3, the implementation assigns each thread block to process a specific atom index i and output indices (L, M) , with warps handling different coupling patterns η for each correlation order ν . The computation can exploit the structure of higher-order correlations by using warp-level butterfly exchange patterns through `__shfl_xor_sync` (❸ in Listing 1) operations to compute tensor products efficiently. In addition, loop unrolling (for example ❹ in Listing 1) eliminate loop overhead in warp-level operations where the number of iterations is known at compile time, allowing better instruction scheduling and register allocation by the compiler.

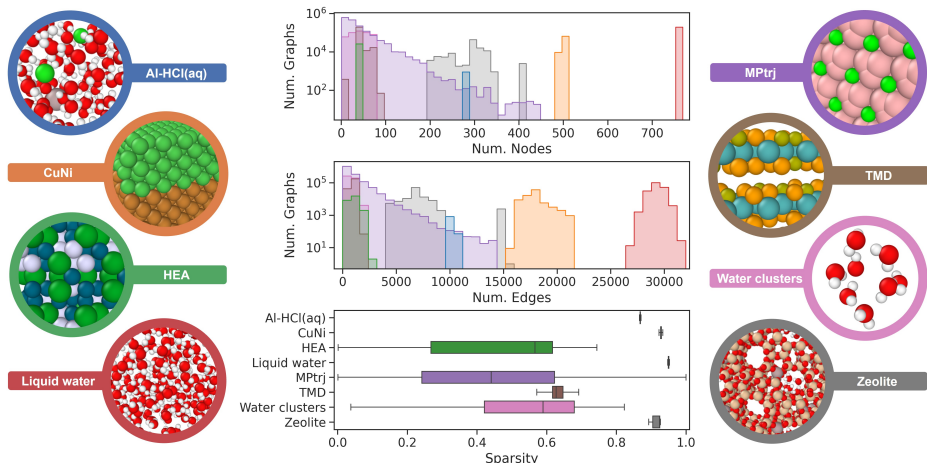


Figure 5: Information on the different chemical systems included in our combined graph dataset: histograms of the vertex and edge counts (in log scale) and sparsity distributions when $r_{\text{cutoff}} = 4.5 \text{ \AA}$.

Table 3: Size of datasets and the range of vertices in each system.

Dataset	Num. Graphs	Proportion	Num. Vertices
Al-HCl(aq)	884	<1%	281–281
CuNi	74335	3%	492–500
HEA	25628	1%	36–48
Liquid water	190267	7%	768–768
MPtrj	1580312	60%	1–444
TMD	219627	8%	16–96
Water clusters	460000	17%	9–75
Zeolite	99770	4%	203–408

5 Evaluation

We investigate the following research questions to evaluate the effectiveness of our proposed optimizations:

- **Effectiveness:** How do our proposed load balancing and kernel optimizations affect model training? We conduct comprehensive ablation studies (Section 5.3) and scaling experiments (Section 5.4) to quantify the individual and combined effects of optimizations.
- **Robustness:** How do our optimizations perform across datasets with varying size and sparsity? We evaluate performance across diverse chemical systems ranging from small molecules to large periodic structures (Section 5.4).
- **Optimal Configuration:** What are the optimal configuration parameters for maximum GPU utilization? We empirically determine the optimal bin capacity that balances computational efficiency with memory constraints (Section 5.5).
- **Performance Analysis:** How do our optimizations affect the computation-communication balance? We answer this via profiling the application.

5.1 Experimental Setup

5.1.1 Dataset. We combine eight datasets with different chemical systems to create a 2.65M graph dataset. Most notable about this collection is the large variation in the number of atoms in each system, which ranges from 1 to 768. When defining molecular graphs, each vertex represents an atom and edges are placed between atom pairs within a radius r_{cutoff} of 4 Å (including distances across periodic boundary conditions). Despite the common r_{cutoff} , the sparsity profiles are highly diverse.

Diversity in size. Figure 5 and Table 3 give information on the included chemical systems and their resulting molecular graphs. The Materials Project Trajectory (MPtrj) database [11], which was used to train MACE-MP-0 [5], comprises 60% of the combined dataset and contains the most diverse set of systems. The second largest dataset in our combined set, comprising 17%, contains molecular dynamics simulations of water clusters [19]. A transition metal dichalcogenide (TMD) dataset comprises 8% [28], and a liquid water dataset comprises 7%. Notably, the liquid water samples are the largest in the combined set with all samples containing 768 atoms. The remaining datasets comprised of zeolite, copper nickel alloy (CuNi) [38], high entropy alloy (HEA) [27], and aluminum ions in aqueous hydrochloric acid (Al-HCl(aq)), each making up <5% of the combined.

Diversity in long/short-range interactions. All systems in the liquid water and CuNi datasets are larger than the largest system in the MPtrj dataset. Notably, these systems represent different types of long-range interactions. The CuNi systems are crystalline with mostly face-centered cubic symmetry; therefore, the ordering is highly regular throughout these systems. In contrast, the liquid water systems are amorphous. Short-range order is present due to hydrogen bonding between neighboring water molecules, but no long-range symmetry is present. However, the absence of symmetry does not imply that long-range interactions are insignificant. Long-range non-bonding interactions and electrostatic effects can still have an effect on system properties.

The MPtrj, TMD, and HEA datasets contain highly ordered crystalline structures, while the water cluster and Al-HCl(aq) datasets contain only short-range ordering. The zeolite dataset has long-range ordering in the zeolite structure, with amorphous groupings of solvent molecules within the zeolite pores.

Dataset variants. The strong scaling experiments were performed based on the combinations mentioned in Table 3, with an approximate size of 2.65 million graphs (samples), with diverse vertex count, edge count and sparsity. For the weak scaling experiments, the combined (large) dataset in Table 3 were split into two additional subsets: small (with $\approx 600,000$ graphs), medium (with ≈ 1.2 million samples).

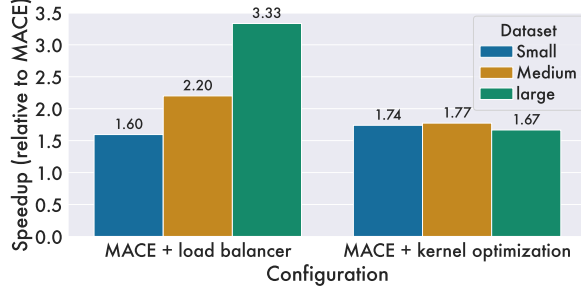


Figure 6: Ablation study results of the proposed optimizations.

5.1.2 Software Configuration. We used PyTorch version 2.3.1 and CUDA version 12.2. The number of workers for the PyTorch DataLoader `num_workers` is set to 12 for pre-processing and data loading (found to be the optimal for our system). The `pin_memory` functionality is enabled to leverage faster CPU to GPU data transfer by using pinned memory. The data is distributed using PyTorch’s DistributedDataParallel (DDP) module, where mini-batches of graphs are distributed across GPUs, with the model copied on each GPU.

5.1.3 Hardware Configuration. The experiments were conducted on NERSC Perlmutter system with 4 NVIDIA A100 GPUs/compute node, and 80 GB main memory/GPU. Each compute node has a single AMD EPYC 7763 CPU, with 64 cores and 256 GB of DDR4 DRAM/CPU. The interconnect is a 3-hop dragonfly topology.

5.2 Hyperparameters

We set the learning rate to 0.005 in our experiments. For baseline minibatching, we specified a batch size of 6 (with systems with large atom counts) to 8 (with systems with small-medium atom counts) without running out of memory and report the average per epoch runtime. We used the Adam optimizer and an exponential moving average learning scheduler, along with the weighted loss function.

Each training was performed with two layers of MACE as demonstrated to be sufficient by [6] due to the application of higher body order to construct equivariant messages. The body order of the message was set to 4. For load balancer, we set the maximum total token size (number of atoms) in a minibatch (bin) to 3072. For training, Float32 datatype was used. The maximum radial cut-off for atom neighborhood (which influences the graph connectivity) was set to 4.5 Å.

The irreducible representations of the messages was set to $128 \times 0e + 128 \times 1o$ according to the notation of the e3nn library [17] (meaning direct sum of two feature: 128 scalar features that are invariant under inversion and 128 vector features that change sign under inversion). Highest ℓ of the spherical harmonics was set to 3. Both the correlation order of each layer and the max L were set to 2. We use 8 Bessel functions for the radial basis.

5.3 Ablation Study

We first conducted an ablation study on the individual impact of the load balancer and the optimization of the symmetric tensor contraction on training MACE with different dataset sizes. For this study, we ran our experiments on 16, 32 and 64 compute nodes for small, medium and large dataset respectively. We measure the speedup of

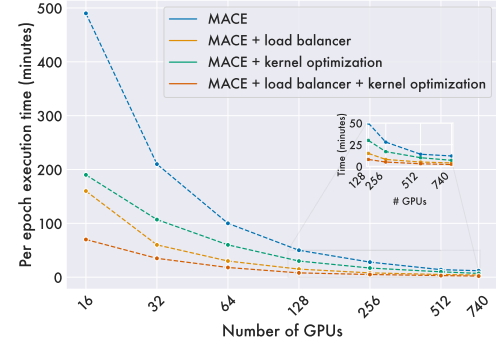


Figure 7: Strong scaling result with 2.6M samples.

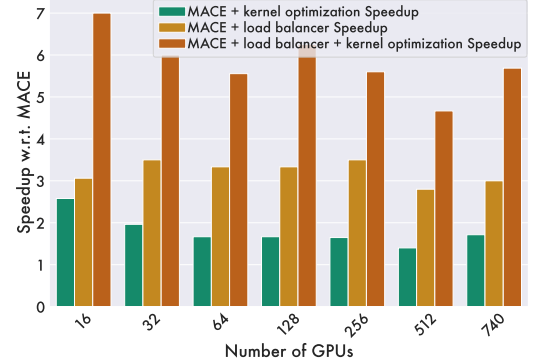


Figure 8: Strong scaling speedup with different optimizations

a model variation as the average per epoch execution time w.r.t. the original MACE implementation. The results are reported in Figure 6. As can be observed from the figure, load balancing has significant impact on the performance of the model. The improvement is particularly evident with large dataset, which has the widest variation in vertex and edge counts – up to 3.3×. With the kernel optimizations, training performance improvements of the model with different datasets are generally the same, up to 1.7×.

5.4 Strong and Weak Scaling Results

5.4.1 Strong Scaling. The strong scaling performance of the MACE model, with and without our proposed optimizations is reported in Figure 7, comparing the per-epoch execution time as the number of GPUs increases. The analysis was done with the combined dataset of ≈ 2.6 million graphs. As the number of GPUs increases, all configurations show improved execution time.

Our proposed load balancing approach provides a significant improvement over both baseline MACE and kernel optimizations. The kernel optimized MACE consistently outperform their baseline MACE counterparts. The combination of kernel optimization with load balancing provides the best performance across all GPU counts tested, achieving a roughly 6× speedup over the original MACE at 740 GPUs (185 compute nodes) (Figure 8).

Using the formula, strong scaling efficiency = $\frac{T_1}{P \times T_P} \times 100\%$, where T_1 is the execution time on the reference number of GPUs, and T_P is the execution time on P GPUs, MACE with load balancer and kernel optimization achieves $(80 / (740 / 16 \times 2)) \times 100\% = 86.5\%$ strong scaling efficiency (calculated from $T_1 \approx 80$ minutes at 16 GPUs to $T_P \approx 2$ minutes at 740 GPUs).

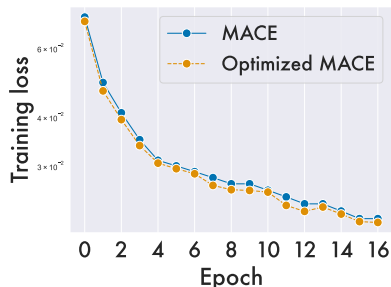


Figure 9: Training loss per epoch for the first 16 epochs.

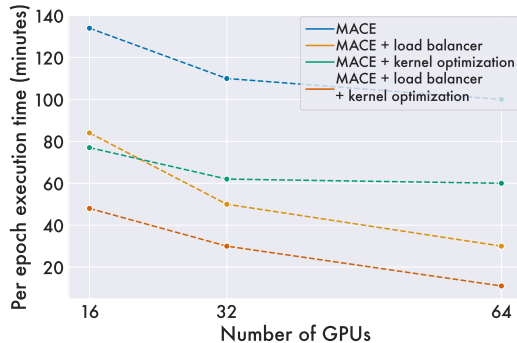


Figure 10: Weak scaling results.

The training loss per epoch with the full dataset on 740 GPUs is reported in Figure 9. The training loss profile of the baseline model and the optimized model follows a similar trajectory (with slight variation in calculated loss), suggesting that the both models evolve their learning in comparable ways.

5.4.2 Weak scaling. Figure 10 shows the weak scaling performance of different MACE configurations, comparing execution time per epoch as the number of GPUs increases while maintaining roughly constant workload (number of graphs) per GPU. The 16-, 32-, and 64-GPU experiments were conducted with the small (0.6 million graphs), medium (1.2 million graphs), and large datasets (2.6 million graphs), respectively. Load balancing provides significant improvement for both MACE and kernel-optimized MACE. Kernel-optimized MACE with load balancer shows the best weak scaling efficiency.

5.5 Empirical Determination of Optimal Bin Capacity and Mini-batch Size

The capacity of a bin (mini-batch) specified in our load-balancing algorithm (Algorithm 1) is GPU-dependent, with the lower bound determined by the largest graph size in the dataset, and the upper bound limited by GPU memory constraints. We aim to saturate the compute on each GPU by packing the bins as much as possible, while constrained by the GPU memory capacity.

Our empirical analysis revealed that, on a single GPU, to saturate the compute, ≈ 400 tokens (total atom count i.e. vertex count of 400) and 800 tokens are the lower bound for the number of tokens that will saturate compute when using Float64 (Figure 11) and Float32 data types respectively. For example, for Float64, as can be observed from Figure 11, with large molecules (500 atoms), doubling the batch size also doubles the execution time. With molecules

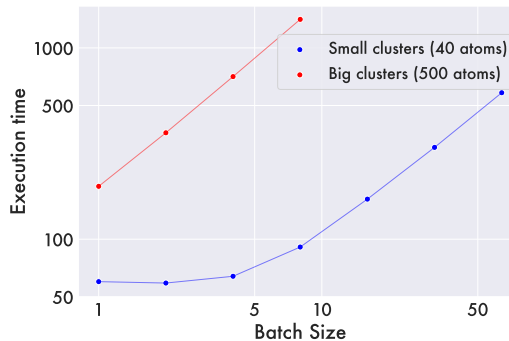


Figure 11: Determining the lower bound on the number of tokens (atoms) in a batch with Float64 data type.

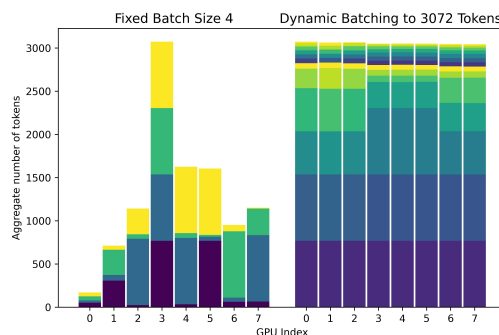


Figure 12: Comparison of the distribution of the samples (graphs) with and without the load balancer. Token count refers to the total number of atoms placed on one GPU.

with small number of atoms, the execution time barely changes, suggesting underutilized computational capacity on the GPU, until the batch size reaches 8. The upper bound is capped by the memory ceiling, roughly 2,000 tokens with Float64 and 4,000 tokens with Float32 settings. Within this range, any value works well, although there is a trade-off to ensure inter-node communication is less frequent while not to have too few batches that may impact the final outcome.

5.6 Workload Characterization

Finally, we investigate the impact of data distribution and kernel optimization on the computation and communication profile of the model. Figure 12 provides a snapshot of how graphs are combined in a mini-batch and distributed across 8 GPUs, with the existing approach (left) and with our approach (right). Each color in the figure represents a graph, with the length of each rectangle proportional to the vertex count in that graph. Specifically, the left part of the figure shows PyTorch’s default *fixed-graph-count* distribution with fixed batch size (graph count) of 4. With this approach, due to the diversity of the vertex count in the molecular graph dataset, each minibatch has widely-varying workload. The runtime is dictated by the straggler on GPU 3. This is evident from Figure 13(a) which demonstrates this imbalance in computation for a fixed batch size.

On the right of Figure 12, we show the distribution of graphs in minibatches with our optimization based approach. Instead of specifying total graph count per mini-batch, we specify a total vertex

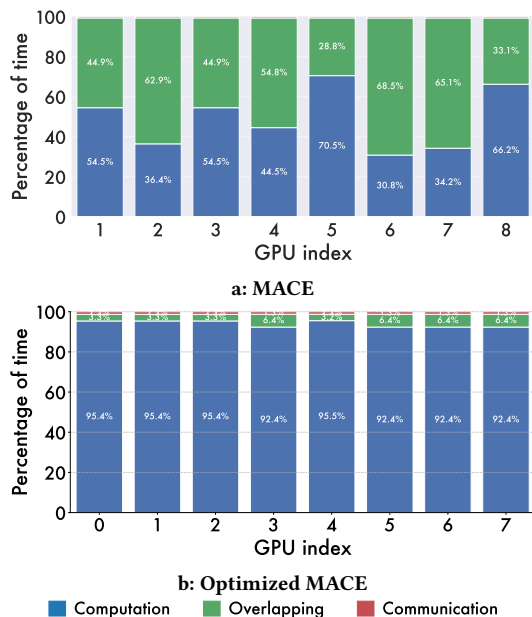


Figure 13: Computation and communication profiles. Here “computation” represents GPU kernel execution time minus overlapping time, while “overlapping” indicates the period where computation and communication occur simultaneously.

count (aggregate number of tokens) of 3072 per minibatch (bin), and our algorithm then selects and places the graphs in mini-batches with an objective of balanced data distribution. We observe that our load balancing approach is able to fit more graphs within the same memory constraint. Notice that with the load balancer, equal distribution of atoms across all GPUs is obtained. More importantly, with load balancing and optimized kernel, the model spends majority of the time in the computation phase (Figure 13(b)), suggesting good throughput, and less time spent in communication. This directly contributes to performance gains.

6 Related Work

This section provides a concise overview of relevant prior work and contextualizes our contribution within the field.

Allen et al. present one of the first approaches to developing CFMs for MLIPs in [2], building on [31]. The authors aggregate seven datasets calculated with varying quantum mechanical levels of theory to train a CFM for small drug-like molecules (<100 atoms). The training process involves repeatedly sampling from disparate datasets and executing a small number of optimization steps with each sampled dataset. While this work provided insight for overcoming challenges associated with training CFMs on aggregated datasets, computational challenges associated with large-scale training were not addressed.

Shoghi et al. introduced an alternative approach, called Joint Multi-domain Pre-Training (JMP) [37] that simultaneously trains on multiple datasets from different chemical domains by treating each dataset as a unique pre-training task within a multi-task framework. The authors combined approximately 120 million systems from the OC20, OC22, ANI-1x, and Transition-1x. While the authors show that fine-tuning the pre-trained model is 12 times faster compared to

training from scratch, pre-training was computationally expensive, requiring 34,400 GPU hours for the large model.

Recently, Pasini et al. introduced the HydraGNN approach to training CFMs over aggregated datasets [33]. HydraGNN uses a multi-headed graph convolutional neural network architecture to employ a mixture of 10 models, including MACE. The authors combined the OC20, OC22, ANI-1x, QM7-X, and MPTrj datasets to obtain approximately 154 million systems and examined a set of invariant and equivariant GNNs. The study demonstrated near-linear strong scaling performance using more than 2,000 GPUs on the Perlmutter supercomputer and 16,000 GPUs on the Frontier supercomputer. *Load imbalance was identified as a significant challenge to scaling performance.* It was attributed to the varying system sizes and was particularly pronounced with equivariant architectures.

7 Conclusion and Future Work

This work presents advancements in the training of chemistry foundation models based on graph neural networks, with a particular focus on models employing equivariant operations grounded in spherical harmonics. Molecular datasets exhibit extreme heterogeneity in terms of system sizes, atom types, and sparsity patterns. This diversity, while essential for capturing the complexity of chemical systems, poses unique challenges for efficient parallel training. We make contributions into two crucial areas: data distribution and model training optimization.

Collectively, these enhancements demonstrate significant performance improvements reducing per-epoch training time from 100 minutes (original MACE) to 18 minutes (optimized MACE) on 64 GPUs, and achieved a 6-fold reduction in per-epoch training time on 740 GPUs from 12 to 2 minutes. Our optimizations target fundamental operations common across equivariant GNN architectures, making them broadly applicable to other models.

One limitation of our developed load balancer is its sacrifice of randomness, which may impact training effectiveness. Additionally, as bottleneck operations in CFMs, such as MACE, are optimized, other operations may become new bottlenecks, requiring further optimization to achieve optimal performance. Regardless of these limitations and necessary future work, the contributions presented here will enable efficient and scalable training of chemistry foundation models.

Acknowledgements

This work was supported by the “Transferring exascale computational chemistry to cloud computing environment and emerging hardware technologies (TEC⁴)” project, which is funded by the U.S. Department of Energy, Office of Science, Office of Basic Energy Sciences, the Division of Chemical Sciences, Geosciences, and Biosciences (under FWP 82037). Pacific Northwest National Laboratory (PNNL) is a multiprogram national laboratory operated for the U.S. Department of Energy (DOE) by Battelle Memorial Institute under Contract No. DE-AC05-76RL0-1830. The authors acknowledge the assistance of Claude (Anthropic) in performing light editorial revisions to the manuscript.

References

- [1] [n. d.]. Bin packing as a scheduling Problem. <https://www.csa.iisc.ac.in/~arindamkhan/seminar/binpacking21/>

- [2] Alice EA Allen, Nicholas Lubbers, Sakib Matin, Justin Smith, Richard Messerly, Sergei Tretiak, and Kipton Barros. 2024. Learning together: Towards foundational models for machine learning interatomic potentials with meta-learning. *npj Computational Materials* 10, 1 (2024), 154. <https://doi.org/10.1038/s41524-024-01339-x>
- [3] Brandon Anderson, Truong Son Hy, and Risi Kondor. 2019. Cormorant: Covariant molecular neural networks. *Advances in neural information processing systems* 32 (2019).
- [4] Ilyes Batatia, Simon Batzner, Dávid Péter Kovács, Albert Musaelian, Gregor NC Simm, Ralf Drautz, Christoph Ortner, Boris Kozinsky, and Gábor Csányi. 2025. The design space of E (3)-equivariant atom-centred interatomic potentials. *Nature Machine Intelligence* (2025), 1–12.
- [5] Ilyes Batatia, Philipp Benner, Yuan Chiang, Alin M Elena, Dávid P Kovács, Janosh Riebesell, Xavier R Advincula, Mark Asta, William J Baldwin, Noam Bernstein, et al. 2023. A foundation model for atomistic materials chemistry. *arXiv preprint arXiv:2401.00096* (2023).
- [6] Ilyes Batatia, David P Kovacs, Gregor Simm, Christoph Ortner, and Gábor Csányi. 2022. MACE: Higher order equivariant message passing neural networks for fast and accurate force fields. *Advances in Neural Information Processing Systems* 35 (2022), 11423–11436.
- [7] Simon Batzner, Albert Musaelian, Lixin Sun, Mario Geiger, Jonathan P Mailoa, Mordechai Kornbluth, Nicola Molinari, Tess E Smidt, and Boris Kozinsky. 2022. E (3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature communications* 13, 1 (2022), 2453.
- [8] Jenna A Bilbrey, Joseph P Heindel, Malachi Schram, Pradipta Bandyopadhyay, Sotiris S Xantheas, and Sutanay Choudhury. 2020. A look inside the black box: Using graph-theoretical descriptors to interpret a Continuous-Filter Convolutional Neural Network (CF-CNN) trained on the global and local minimum energy structures of neutral water clusters. *The Journal of Chemical Physics* 153, 2 (2020).
- [9] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).
- [10] Benjamin Coors, Alexandru Paul Condurache, and Andreas Geiger. 2018. Spherenet: Learning spherical representations for detection and classification in omnidirectional images. In *Proceedings of the European conference on computer vision (ECCV)*. 518–533.
- [11] Bowen Deng, Peichen Zhong, Kyujung Jun, Janosh Riebesell, Kevin Han, Christopher J Bartel, and Gerbrand Ceder. 2023. CHGNet as a pretrained universal neural network potential for charge-informed atomistic modelling. *Nature Machine Intelligence* 5, 9 (2023), 1031–1041.
- [12] Ralf Drautz. 2019. Atomic cluster expansion for accurate and transferable interatomic potentials. *Physical Review B* 99, 1 (2019), 014104.
- [13] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [14] Michael R. Garey and David S. Johnson. 1979. *Computers and intractability: A guide to the theory of NP-completeness*. W.H. Freeman and Company.
- [15] Johannes Gasteiger, Florian Becker, and Stephan Günnemann. 2021. Gemnet: Universal directional graph neural networks for molecules. *Advances in Neural Information Processing Systems* 34 (2021), 6790–6802.
- [16] Johannes Gasteiger, Janek Groß, and Stephan Günnemann. [n. d.]. Directional Message Passing for Molecular Graphs. In *International Conference on Learning Representations*.
- [17] Mario Geiger, Tess Smidt, Alby M., Benjamin Kurt Miller, Wouter Boomsma, Bradley Dice, Kostiantyn Lapchevskiy, Maurice Weiler, Michał Tyszkiewicz, Simon Batzner, Dylan Madiseti, Martin Uhrin, Jes Frellsen, Nuri Jung, Sophia Sanborn, Mingjian Wen, Josh Rackers, Marcel Rød, and Michael Bailey. 2022. *Euclidean neural networks: e3nn*. <https://doi.org/10.5281/zenodo.6459381>
- [18] Yidong Gong and Pradeep Kumar. 2024. GNNOne: A Unified System Optimizations for GNN Kernels. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*. 15–27.
- [19] Hatem Helal, Jesun Firoz, Jenna A Bilbrey, Henry Sprueill, Kristina M Herman, Mario Michael Krell, Tom Murray, Manuel Lopez Roldan, Mike Kraus, Ang Li, et al. 2024. Acceleration of Graph Neural Network-Based Prediction Models in Chemistry via Co-Design Optimization on Intelligence Processing Units. *Journal of Chemical Information and Modeling* 64, 5 (2024), 1568–1580.
- [20] Silvan Käser, Luis Itza Vazquez-Salazar, Markus Meuwly, and Kai Töpfer. 2023. Neural network potentials for chemistry: concepts, applications and prospects. *Digital Discovery* 2, 1 (2023), 28–58.
- [21] Tsz Wai Ko and Shyue Ping Ong. 2023. Recent advances and outstanding challenges for machine learning interatomic potentials. *Nature Computational Science* 3, 12 (2023), 998–1000.
- [22] Bernhard Korte and Jens Vygen. 2012. *Combinatorial Optimization*. Algorithms and Combinatorics, Vol. 21. Springer Berlin Heidelberg, Berlin, Heidelberg. <https://doi.org/10.1007/978-3-642-24488-9>
- [23] Dávid Péter Kovács, Ilyes Batatia, Eszter Sára Arany, and Gábor Csányi. 2023. Evaluation of the MACE force field architecture: From medicinal chemistry to materials science. *The Journal of Chemical Physics* 159, 4 (2023).
- [24] Maksim Kulichenko, Justin S Smith, Benjamin Nebgen, Ying Wai Li, Nikita Fedik, Alexander I Boldyrev, Nicholas Lubbers, Kipton Barros, and Sergei Tretiak. 2021. The rise of neural networks for materials and chemical dynamics. *J. Phys. Chem. Lett.* 12, 26 (2021), 6227–6243.
- [25] Mingyi Li, Junmin Xiao, Kewei Zhang, Zhiheng Lin, Chaoyang Shui, Ke Meng, Zehua Wang, Yunfei Pang, and Guangming Tan. 2024. A Coordinated Strategy for GNN Combining Computational Graph and Operator Optimizations. In *Proceedings of the 38th ACM International Conference on Supercomputing*. 460–472.
- [26] Yi-Lun Liao and Tess Smidt. [n. d.]. Equiformer: Equivariant Graph Attention Transformer for 3D Atomistic Graphs. In *The Eleventh International Conference on Learning Representations*.
- [27] Nataliya Lopanitsyna, Guillaume Fraux, Maximilian A Springer, Sandip De, and Michele Ceriotti. 2023. Modeling high-entropy transition metal alloys with alchemical compression. *Physical Review Materials* 7, 4 (2023), 045802.
- [28] Scott E Muller, Micah P Prange, Zexi Lu, W Steven Rosenthal, and Jenna A Bilbrey. 2023. An open database of computed bulk ternary transition metal dichalcogenides. *Scientific Data* 10, 1 (2023), 336.
- [29] Albert Musaelian, Simon Batzner, Anders Johansson, Lixin Sun, Cameron J Owen, Mordechai Kornbluth, and Boris Kozinsky. 2023. Learning local equivariant representations for large-scale atomistic dynamics. *Nature Communications* 14, 1 (2023), 579.
- [30] Samuel P Niblett, Panagiotis Kourtis, Ioan-Bogdan Magdău, Clare P Grey, and Gábor Csányi. 2024. Transferability of datasets between Machine-Learning Interaction Potentials. *arXiv preprint arXiv:2409.05590* (2024).
- [31] Alex Nichol, Joshua Achiam, and John Schulman. 2018. On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999* (2018).
- [32] Jie Pan. 2021. Scaling up system size in materials simulation. *Nature Computational Science* 1, 2 (2021), 95–95.
- [33] Massimiliano Lupo Pasini, Jong Youl Choi, Kshitij Mehta, Pei Zhang, David Rogers, Jonghyun Bae, Khaled Z Ibrahim, Ashwin M Aji, Karl W Schulz, Jorda Polo, et al. 2024. Scalable Training of Trustworthy and Energy-Efficient Predictive Graph Foundation Models for Atomistic Materials Modeling: A Case Study with HydraGNN. *arXiv preprint arXiv:2406.12909* (2024).
- [34] Dominic Poethokaran and Yoichi Shimakawa. 2010. A study on evaluation of algorithms for multi-objective Bin Packing Problems. In *The 40th International Conference on Computers & Industrial Engineering*. IEEE, 1–6.
- [35] Victor Garcia Satorras, Emiel Hoogeboom, and Max Welling. 2021. E (n) equivariant graph neural networks. In *International conference on machine learning*. PMLR, 9323–9332.
- [36] Kristof Schütt, Pieter-Jan Kindermans, Huziel Enoc Saucedo Felix, Stefan Chmiela, Alexandre Tkatchenko, and Klaus-Robert Müller. 2017. SchNet: A continuous-filter convolutional neural network for modeling quantum interactions. *Advances in neural information processing systems* 30 (2017).
- [37] Nima Shoghi, Adeesh Kolluru, John R Kitchen, Zachary Ward Ulissi, C Lawrence Zitnick, and Brandon M Wood. 2024. From Molecules to Materials: Pre-training Large Generalizable Models for Atomic Property Prediction. In *The Twelfth International Conference on Learning Representations*.
- [38] Henry W Sprueill, Jenna A Bilbrey, Qin Pang, and Peter V Sushko. 2023. Active sampling for neural network potentials: Accelerated simulations of shear-induced deformation in Cu–Ni multilayers. *The Journal of Chemical Physics* 158, 11 (2023).
- [39] Nathaniel Thomas, Tess Smidt, Steven Kearnes, Lusann Yang, Li Li, Kai Kohlhoff, and Patrick Riley. 2018. Tensor field networks: Rotation-and translation-equivariant neural networks for 3d point clouds. *arXiv preprint arXiv:1802.08219* (2018).
- [40] Guanjie Wang, Changrui Wang, Xuanguang Zhang, Zefeng Li, Jian Zhou, and Zhimei Sun. 2024. Machine learning interatomic potential: Bridge the gap between small-scale models and realistic device-scale simulations. *Science* 27, 5 (2024).
- [41] Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. 2024. Scaling New Heights: Transformative Cross-GPU Sampling for Training Billion-Edge Graphs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [42] Anthony Zee. 2016. *Group theory in a nutshell for physicists*. Vol. 17. Princeton University Press.
- [43] Da Zheng, Chao Ma, Minjie Wang, Jinjing Zhou, Qidong Su, Xiang Song, Quan Gan, Zheng Zhang, and George Karypis. 2020. DistDGL: Distributed graph neural network training for billion-scale graphs. In *2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*. IEEE, 36–44.
- [44] Roman Zubatyuk, Justin S Smith, Benjamin T Nebgen, Sergei Tretiak, and Olexandr Isayev. 2021. Teaching a neural network to attach and detach electrons from molecules. *Nature communications* 12, 1 (2021), 4870.